

অলীকবচন

Bengali LLM Hallucination Detection Challenge

A Three-Stage Hybrid Pipeline

Symbolic math reasoning + retrieval-augmented evidence + a fine-tuned verifier LLM

Team : hawkeye

The Challenge

Given a Bengali question, an optional context passage, and a candidate answer — decide if the answer is faithful or hallucinated.

- **Domain mix**

Some rows are precise math word problems; others are open-domain factual QA — one verifier style doesn't fit both.

- **Sparse context**

Many rows arrive with no context passage at all, so world knowledge must be retrieved, not assumed.

- **Bengali numerals**

Answers mix Bengali digits (০-৯), Arabic digits, and spelled-out number words (পঁচিশ, একশ) that must all resolve to the same value.

- **Scale**

~5,000 test rows must be scored end-to-end in under 5 hours on shared GPUs.

SAMPLE ROW

প্রশ্ন (Question)

অব্র কিবোর্ড কে উদ্ভাবন করেন?

উত্তর (Candidate Answer)

মেহদী হাসান খান

MODEL MUST OUTPUT

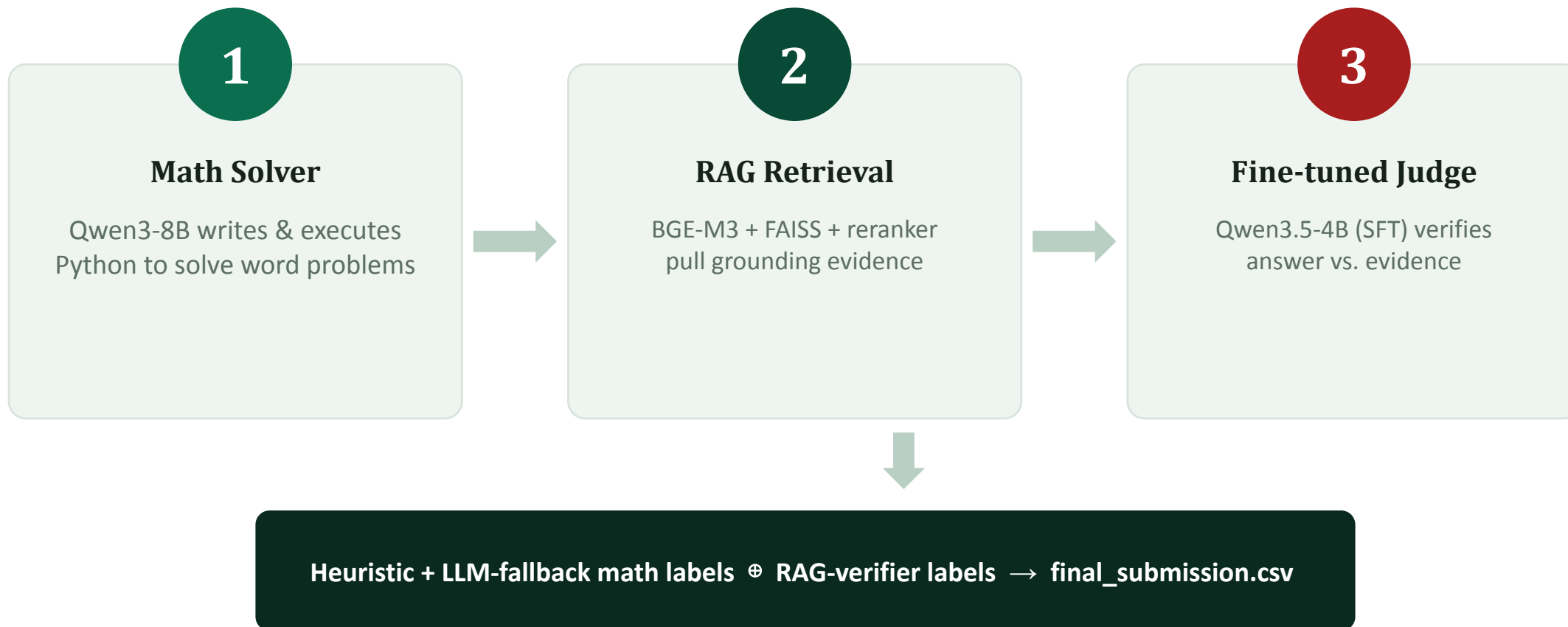
true

false

Label = 1 (faithful) vs 0 (hallucinated), one prediction per row.

Solution Overview

Route each row to the cheapest reliable verifier: solve it symbolically if possible, otherwise ground it in evidence and let a fine-tuned judge decide.



1

Stage 1 — SC-TIR Math Verification Engine

Bengali math word problems are detected up front and routed to a code-writing model instead of a free-text judge.

1 Detect

A heuristic (`is_math_problem`) flags rows that are numeric word problems, leaving the rest for Stages 2–3.

2 Generate code

Qwen3-8B (served with vLLM, tensor-parallel across 2 GPUs) is prompted to translate the Bengali problem and write a short Python script that saves the result to `final_answer`.

3 Execute

Every generated script runs in parallel (`ThreadPoolExecutor`) in a sandboxed subprocess, so a single bad script can't stall the batch.

4 Vote

With self-consistency sampling, numeric outputs across samples are clustered within a tolerance (`numeric_tol = 1e-2`) and the majority value is elected as the answer.

Config: `model = Qwen3-8B · temperature = 0.1 · max_tokens = 2048 · dtype = float16 · tensor_parallel_size = 2`

1

Stage 1 — Scoring the Math Answer

The elected numeric answer is compared against the candidate answer text using symbolic checks first, then an LLM only when needed.

Path A · Symbolic Heuristic

Bengali number parsing:

Digits (০-৯) are mapped to ASCII; a custom dictionary resolves number words (পঁচিশ = 25) and scale words (শত, হাজার, লক্ষ, কোটি).

Candidate extraction:

Every numeric value mentioned in the candidate answer text is pulled out as a match candidate.

Tolerant match:

The elected code answer is compared to each candidate within a relative/absolute tolerance — a hit resolves the label instantly, no LLM call needed.

Path B · LLM Judge Fallback

When triggered:

The code failed to execute, or no numeric candidate matched within tolerance.

Unit-aware comparison:

The same Qwen3-8B model is prompted to reason about whether values match, explicitly accounting for units like দিন (days), টাকা (taka), বছর (years), জন (people).

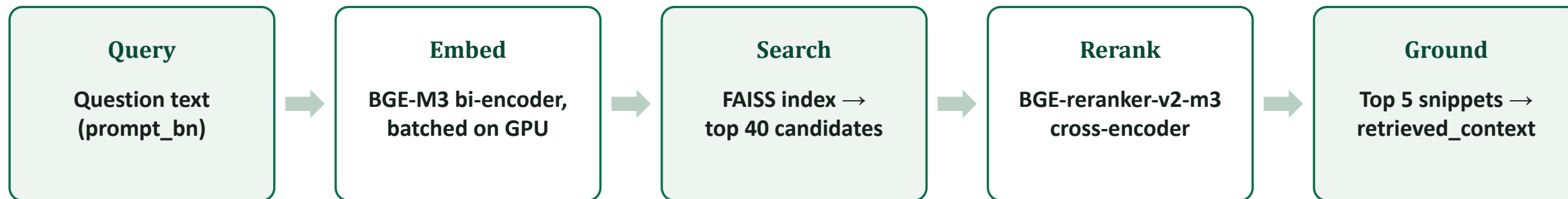
Structured output:

The model reasons freely, then must end with a single parseable line: “*VERDICT: 1*” or “*VERDICT: 0*”.

2

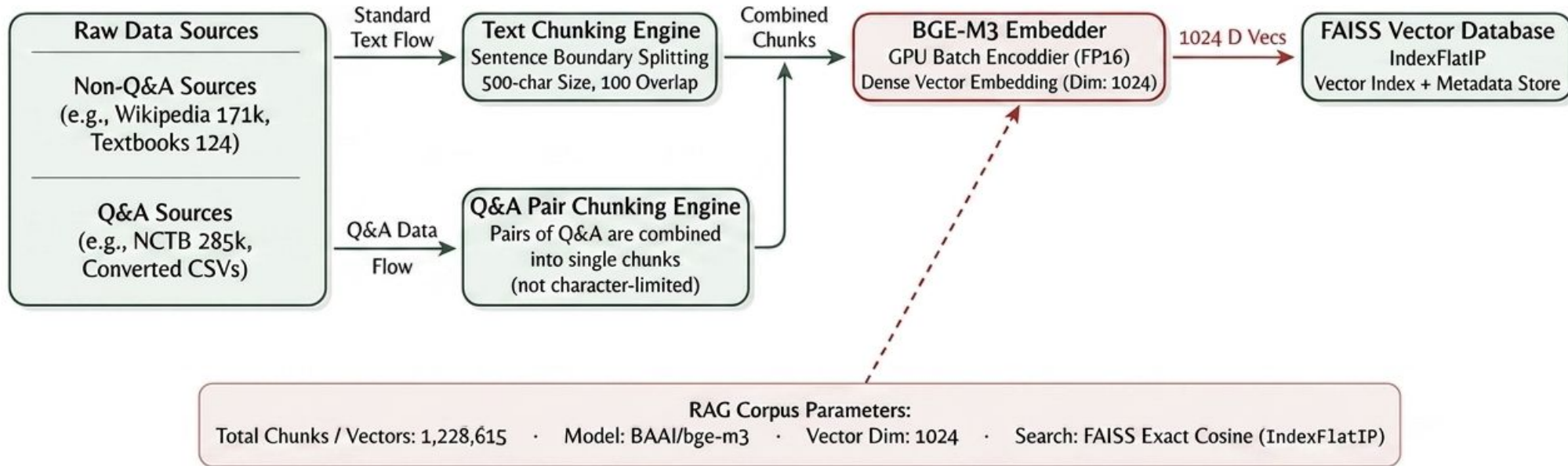
Stage 2 — Retrieval-Augmented Evidence

Every remaining row is grounded against a pre-built Bengali knowledge corpus before it reaches the verifier.



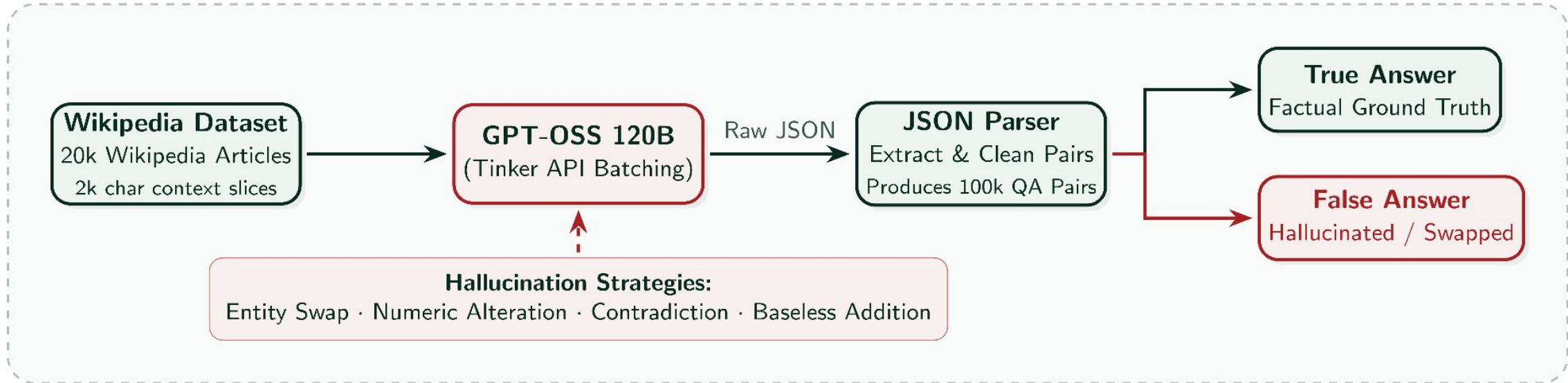
- **Dense retrieval:** Query and Corpus chunks are embedded with BAAI/bge-m3 and compared by cosine similarity (inner-product FAISS search).
- **Two-stage retrieval:** A cheap bi-encoder casts a wide net (40 candidates), then a precise cross-encoder reranker (bge-reranker-v2-m3) rescoring picks the final top 5.
- **Always-on option:** A flag (RETRIEVE_WITH_CONTEXT) lets retrieval run for every row, not only ones missing a context passage — adding corroborating evidence even when context exists.

Rag corpus & Vector Embedding Generation

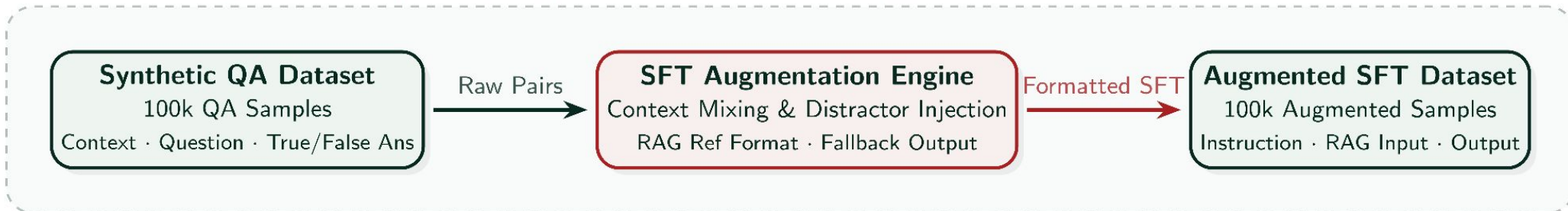


Training (Dataset generation)

Synthetic Data Generation Pipeline (Stage 1: Raw QA Generation)

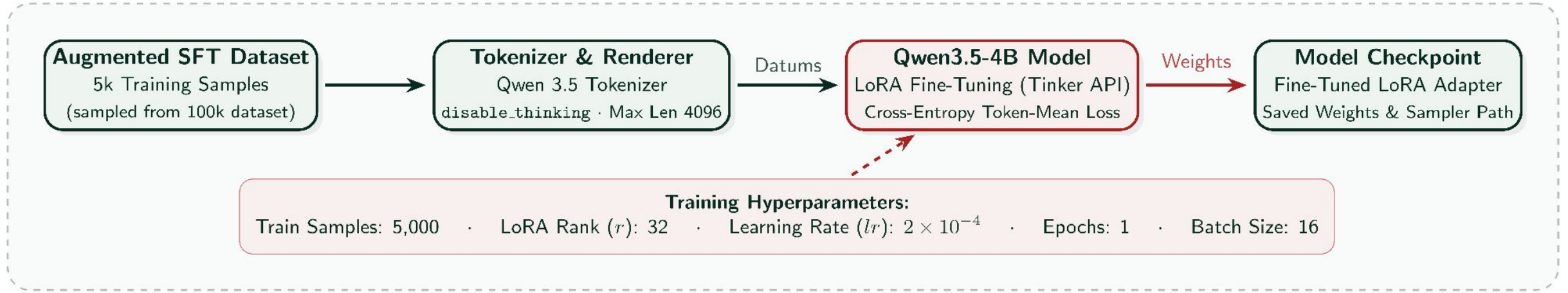


SFT Dataset Augmentation Pipeline (Stage 2: RAG & Context Mixing)



Training (SFT)

SFT Training Pipeline (Stage 3: LoRA Fine-Tuning)



3

Stage 3 — Fine-tuned Fact-Checking Judge

A fine-tuned Qwen3.5-4B model makes the final true/false call, using both the row's own context and the retrieved evidence.

1

Fine-tuned model

The merged Qwen3.5-4B is served with vLLM — specialized for this fact-checking format rather than a generic chat model.

2

Few-shot fact-checking prompt

System instructions plus worked examples teach the model to compare the Answer strictly against Context (and Relevant Info when context is absent) and reply with only “true” or “false”.

3

Token-budget-safe context

Retrieved evidence is split into snippets and greedily packed into the prompt; if it still doesn't fit, a binary-search clip trims text while guaranteeing at least one snippet always survives — no row is ever skipped.

4

Confidence via logprobs

Rather than trusting the sampled token, the top-20 logprobs at the first output position are scanned for every spelling of “true”/“false”, then softmax-normalized into a calibrated probability pair.

Combining Predictions

Two disjoint prediction sets — resolved by the math branch and everything else scored by the RAG-grounded judge — are concatenated and sorted back into submission order.

submission_math.csv

Rows resolved in Stage 1: symbolic heuristic matches plus the LLM judge fallback for unmatched math problems.

+

submissionVLLM2.csv

Every remaining row: RAG-retrieved evidence scored by the fine-tuned Qwen3.5-4B verifier.



final_submission.csv

concat → sort_values("id") → keep [id, label]

Every test row receives exactly one label, produced by whichever stage could resolve it with the highest confidence.

Why the Hybrid Approach Works

- **Right tool per row**

Math questions get exact, verifiable answers from code execution instead of a language model's fuzzy arithmetic.

- **Grounded, not guessed**

Two-stage retrieval (bi-encoder + reranker) supplies precise evidence before any factual judgment is made.

- **Specialized judge**

A fine-tuned, logprob-calibrated verifier replaces a generic “ask an LLM” prompt with a model tuned for this exact task.

- **Nothing falls through**

Fallback paths and guaranteed-fit prompting mean every one of ~5,000 rows gets a confident, on-time label.

What didn't work

Other Pipeline Evaluation across LLM Backbones and Verification Paradigms

Pipeline Architecture	Backbone LLM	Training & Feature Configuration	Public F1	Private F1
Direct SFT Classifier (Best)	Qwen3.5-4B	5k Augmented + RAG + SC-TIR Math	0.901	0.903
Direct SFT Classifier (9B Scale)	Qwen3.5-9B	10k Augmented + RAG + SC-TIR Math	0.899	0.894
Untrained Few-Shot Baseline	Gemma-4-26B	Few-Shot + RAG + SC-TIR Math	0.894	0.883
SFT Hunter + Fine-Tuned NLI Judge	Qwen3.5-4B	5k Augmented + Fine-Tuned NLI Judge	0.868	0.876
SFT Hunter + Pretrained NLI	Qwen3.5-4B	5k Augmented + Pretrained NLI Classifier	0.869	0.859
Few-Shot Baseline (No RAG/Math)	Gemma-31B	Few-Shot (Without RAG & Math Solver)	0.834	0.843
SFT on Raw Synthetic (100k)	Qwen3.5-9B	100k Un-augmented SFT (qwen_train) + RAG	0.800	0.795
One-Shot Baseline (No RAG/Math)	Gemma-31B	1-Shot (Without RAG & Math Solver)	0.694	0.683



Thank You